Zero-Cost Native iOS 26 Music Companion Architecture: Engineering Blueprint for a Windows-to-iPhone Pipeline

Executive Feasibility and System Boundaries

Developing a private, personal-use music application for an iPhone 14 Pro running iOS 26.5 from a Windows 11 workstation at a strict €0.00 ($0.00) total lifecycle cost—without access to physical macOS hardware, paid cloud macOS rentals, or the $99/year Apple Developer Program—is fully achievable[1]. Reaching production-grade stability under these constraints requires an architecture designed around three non-negotiable technical boundaries enforced by Apple and Spotify in 2026[1].

First, the compilation and code-signing pipeline must decouple source editing from macOS execution[1]. Source code written in Native Swift 6 and SwiftUI targeting the iOS 26 SDK can be authored in Visual Studio Code on Windows 11, compiled into unsigned .ipa bundles at zero cost using GitHub Actions macos-latest (Xcode 26) runners on a public repository, and signed locally on the target iPhone 14 Pro via SideStore and LocalDevVPN using a free Personal Team Apple ID[1]. Because a public repository is utilized to unlock unlimited macOS runner minutes, the codebase must enforce a strict zero-secret policy where client identifiers and OAuth credentials never touch version control, residing instead in the iPhone's hardware-backed Keychain after on-device onboarding[1].

Second, the audio architecture must respect Spotify's playback ownership model and DRM constraints[1]. Under Spotify's Developer Terms and the February/March 2026 Development Mode policies, third-party applications are prohibited from decrypting or streaming raw Spotify audio frames into custom audio buffers[1]. Consequently, the official Spotify iOS application serves as the local background audio engine holding the active AVAudioSession, while the custom application operates as a low-latency Hybrid Controller and Visualizer[1]. This hybrid engine pairs the Spotify iOS App Remote SDK (SPTAppRemote) for sub-15ms local inter-process communication (IPC) with the Spotify Web API (via OAuth 2.0 PKCE) for catalog search, library mutation, and remote Spotify Connect orchestration[1]. Crucially, Spotify's 2026 Development Mode rules mandate that the developer account owning the Web API Client ID maintain an active Spotify Premium subscription; if the owner account lapses to a free tier, all Web API requests fail with HTTP 403 Forbidden[1].

Third, system integration must account for iOS audio session arbitration and the free Apple ID bundle identifier ceiling[1]. Because the background Spotify process owns the active AVAudioSession, the iOS mediaserverd daemon grants Spotify exclusive ownership of the system Lock Screen MPNowPlayingInfoCenter card and the primary Dynamic Island audio pill[1].

Furthermore, a free Personal Team Apple ID permits a maximum of three active sideloaded application or extension bundle identifiers concurrently[1]. With SideStore occupying one slot and the primary music companion application occupying the second, adding a WidgetKit or ActivityKit Live Activity extension consumes the third and final slot[1]. The optimal engineering solution ships a Single-Bundle-ID Native SwiftUI application by default, isolating any WidgetKit extension behind an optional compile-time flag (ENABLE_LIVE_ACTIVITY_EXTENSION)[1].

Capability Category	**Operational Status**	**Technical Reality on iOS 26.5 (Free Apple ID + Windows 11)**	**Primary Verification Source & Status (Oct 2026)**
Native iOS 26 Liquid Glass UI	Confirmed	Full access to public SwiftUI APIs (GlassEffectContainer, .glassEffect(), .glassEffectID, .buttonStyle(.glass)) compiled via Xcode 26 on GitHub Actions[1].	Apple SwiftUI Documentation (WWDC25 / iOS 26 SDK) — Active[1]
120Hz ProMotion Synced Lyrics	Confirmed	Achieved via a local CADisplayLink monotonic clock interpolation engine anchored to Spotify playback timestamps, rendering LRC and Enhanced LRC/TTML word-synced lyrics at 120fps[1].	LRCLIB API (lrclib.net/docs, ~413ms avg latency, €0, no key) — Active[1]
Adaptive Album Atmosphere	Confirmed	Local $64 \times$ vImage / CoreImage LAB k-means palette extraction driving a native Metal-backed	Apple Accelerate & SwiftUI MeshGradient Frameworks — Active[1]

		SwiftUI MeshGradient ambient background at 60fps[1].	
Spotify Search, Library & Control	Confirmed	Supported via OAuth 2.0 PKCE + Web API (GET /search, GET/PUT/DELETE /me/library, /playlists/{id}/items) + iOS App Remote SDK (SPTAppRemote); requires a Premium owner account[1].	Spotify Web API (Feb/Mar 2026 rules) & spotify/ios-sdk — Active[1]
€0 Windows-to-iPhone Build & Refresh	Confirmed	Public GitHub repository grants unlimited free macos-latest Xcode 26 build minutes; SideStore + LocalDevVPN refreshes the 7-day certificate on-device over Wi-Fi without PC tethering after pairing[1].	GitHub Actions Runners & SideStore (docs.sidestore.io) — Active[1]
Direct Raw Spotify Audio Streaming	Unavailable	Blocked by Spotify DRM and Developer Policy; audio outputs exclusively through the installed Spotify iOS application running in the background[1].	Spotify Developer Terms & Media Streaming Policy — Enforced[1]

Overriding System Lock Screen Player	Unavailable	iOS binds MPNowPlayingInfo Center and Lock Screen full-screen artwork exclusively to the process holding the active AVAudioSession (Spotify)[1].	Apple MediaPlayer Framework (MPNowPlayingInfo Center) — Enforced[1]
Dynamic Island / Live Activities	Restricted	Supported via local ActivityKit updates, but requires a separate WidgetKit Extension Bundle ID, consuming 1 of 3 free Apple ID sideload slots; isolated behind an optional build flag[1].	Apple ActivityKit & Free Provisioning Profile Quotas — Restricted[1]

Spotify 2026 Policy Landscape and Dual-Engine Hybrid Architecture

The 2026 Development Mode Regulatory Shift

Effective March 9, 2026, following its February 6, 2026 developer policy announcement, Spotify restructured the technical capabilities and access rules for all applications operating in Development Mode[1]. These regulatory adjustments directly dictate how a private companion application must structure its network layer and data models[1].

First, Spotify instituted a mandatory Premium subscription requirement for the application owner[1]. The primary Spotify account used to register the client_id in the Spotify Developer Dashboard must maintain an active Spotify Premium plan (Individual, Duo, Family, or Student)[1]. If the owner account transitions to Spotify Free, the API gateway rejects all authenticated Web API requests under that client_id with HTTP 403 Forbidden, regardless of the end-user's subscription tier[1]. Additionally, Development Mode enforces a strict five-user allowlist cap, requiring every authenticating user to be explicitly registered by email in the Developer Dashboard[1].

Second, Spotify executed widespread schema pruning and endpoint consolidation across the Web API in Development Mode[1]. Within SimplifiedPlaylist and FullPlaylist payloads, the legacy tracks object was renamed to items (accessed via /v1/playlists/{id}/items), and within each PlaylistItem container, the nested track property was renamed to item[1]. Furthermore, batch

multi-ID lookup endpoints and artist_top_tracks were removed from Development Mode access, while metadata fields including available_markets, external_ids (which previously supplied ISRC codes), linked_from, popularity, followers, country, email, and product were stripped from FullTrack, FullAlbum, FullArtist, and PrivateUser objects[1]. Library mutations are now unified under GET /v1/me/tracks for reads and PUT / DELETE /v1/me/library?uris=... for modifications, while catalog search (GET /v1/search) is capped at a maximum pagination limit of 10 items per request[1].

The removal of external_ids (specifically ISRC identifiers) from track payloads introduces a significant second-order engineering consequence: the lyrics subsystem can no longer rely on universal recording codes for deterministic database lookups[1]. Instead, track resolution against external lyric repositories must utilize composite metadata signatures combining normalized track_name, artist_name, album_name, and duration_ms within a $\pm 2.0\text{s}$ tolerance window[1].

Feature Domain	Supported?	Account & Application State Requirements	2026 API / SDK Technical Limitations
Search Spotify (Tracks, Albums, Artists, Playlists)	Yes	Valid OAuth 2.0 PKCE Access Token; Client ID owner must hold active Spotify Premium[1].	Maximum limit=10 results per page in Development Mode; popularity and external_ids (ISRC) fields omitted from track responses[1].
Control Playback (Play, Pause, Skip, Previous, Seek, Shuffle, Repeat, Volume)	Yes	Spotify Premium + official Spotify iOS app installed and authenticated on the iPhone 14 Pro[1].	Local SPTAppRemote IPC requires waking the Spotify app via URL scheme if terminated; Web API /me/player/* requires an active Connect session[1].
Read Playback Progress & Active State	Yes	Active session via SPTAppRemotePlayerAPI delegate or Web API GET /v1/me/player[1].	Web API relies on HTTP polling (polling faster than 1 req/3s risks HTTP 429); requires SPTAppRemote

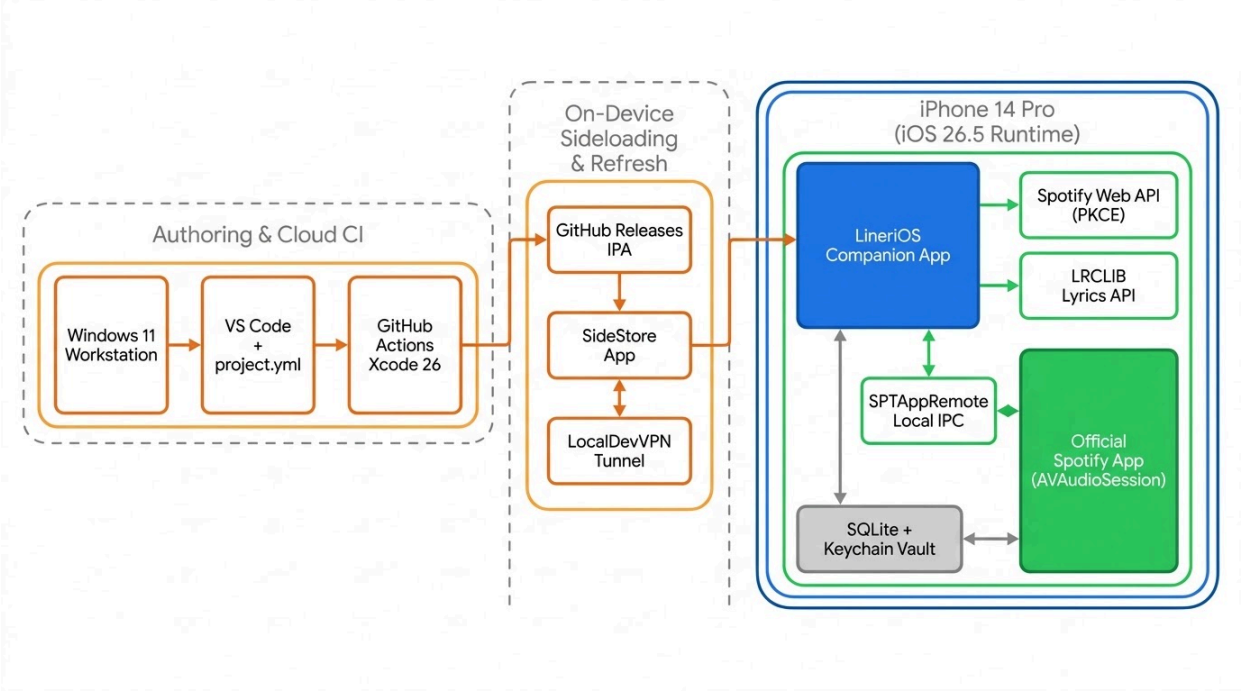
			push events + local CADisplayLink interpolation[1].
Background Playback	Yes (Delegated)	Official Spotify iOS app running in background holding the active AVAudioSession[1].	Companion app outputs no audio; force-killing the Spotify app halts playback until woken via spotify: URI scheme[1].
Display Synchronized Lyrics	Yes (External)	Track title, artist, album, and duration_ms extracted from active Spotify metadata[1].	Spotify exposes no public lyrics API; lyrics must be resolved via LRCLIB or local .lrc/.ttml imports using metadata signature matching[1].
Native Lock Screen Controls	Handled by Spotify	Official Spotify iOS app active in background[1].	iOS binds MPNowPlayingInfo Center to the active audio process; Lock Screen commands route to Spotify and broadcast back via SPTAppRemote[1].

The Dual-Engine Hybrid Integration Model

Relying solely on the Spotify Web API for a real-time lyric visualizer fails because HTTP polling introduces network latency (200–600ms) and rapidly exhausts Spotify's rolling 30-second rate-limit window, triggering HTTP 429 Too Many Requests lockouts[1]. Conversely, relying solely on the Spotify iOS App Remote SDK (SPTAppRemote) is insufficient because the local IPC protocol lacks full catalog search and flexible playlist inspection capabilities[1].

To achieve sub-15ms transport responsiveness alongside full library access, the architecture implements a Dual-Engine Hybrid model sharing a single OAuth 2.0 Authorization Code with PKCE credential set[1]. The Local Control Plane utilizes SPTAppRemote (SpotifyiOS.xcframework) to establish a direct inter-process communication channel (spotify-action://) with the official Spotify application installed on the iPhone 14 Pro[1]. This local

socket executes Play, Pause, Skip, and Seek commands in under 15ms and receives push-based SPTAppRemotePlayerState delegate callbacks whenever playback state or track position changes, consuming zero Web API quota during normal listening[1]. Complementing the local socket, the Cloud Metadata and Connect Plane leverages the Spotify Web API with PKCE tokens to execute debounced catalog searches (limit=10), fetch user playlists via /v1/playlists/{id}/items, mutate saved library items via /v1/me/library, and automatically assume transport control via /v1/me/player whenever playback is handed off to an external Spotify Connect target such as a Windows 11 desktop speaker system[1].



iOS 26 Audio Session Arbitration and Free Apple ID Entitlement Limits

The Audio Session Ownership Conflict

On iOS 26.5, system-wide media controls—comprising the Lock Screen media player card, full-screen Lock Screen album artwork, Control Center transport cluster, hardware volume/headset event routing (MPRemoteCommandCenter), and the primary Dynamic Island Now Playing activity—are strictly arbitrated by the system audio server (mediaserverd)[1]. The operating system grants ownership of MPNowPlayingInfoCenter exclusively to the process currently holding an active, playing AVAudioSession configured with the .playback category[1]. If a companion application attempts to hijack the Lock Screen by activating its own non-mixable .playback audio session (for instance, by looping an inaudible audio buffer), iOS immediately interrupts and pauses the official Spotify application's audio stream[1]. Conversely, if the companion application configures its session with the .mixWithOthers option to avoid interrupting Spotify, mediaserverd ignores the companion application's MPNowPlayingInfoCenter metadata dictionary and continues displaying the primary

non-mixable audio producer (Spotify)[1]. Furthermore, Apple Music's full-screen looping HLS video Lock Screen canvases rely on private system entitlements (com.apple.mediaremote.*) that cannot be provisioned outside Apple's first-party binaries[1].

The robust architectural solution embraces this system boundary rather than fighting it: the official Spotify iOS application naturally retains ownership of the system Lock Screen and Control Center audio session when the device is locked[1]. Hardware button presses, AirPods stem squeezes, and Lock Screen transport taps route natively to Spotify, which immediately broadcasts playerStateDidChange events back to the companion application over SPTAppRemote[1]. When the iPhone 14 Pro is unlocked or docked on a MagSafe desk charger (with UIApplication.shared.isIdleTimerDisabled = true engaged), the companion application takes over the foreground viewport to deliver the full 120Hz ProMotion Liquid Glass, Adaptive Atmosphere, and Syllable-Synced Lyrics experience[1].

Free Personal Team Provisioning Quotas

Sideloading under a free Personal Team Apple ID introduces two strict provisioning ceilings enforced by Apple's signing servers: a maximum of 3 active installed application or extension bundles on a physical device at any moment, and a maximum of 10 App ID registrations per rolling 7-day window[1]. Because an ActivityKit Live Activity or WidgetKit component compiles as a separate App Extension target (com.personal.linerios.widget) requiring its own explicit Bundle ID and an App Group entitlement, enabling a Dynamic Island widget consumes a full slot (SideStore [1] + LineriOS Main App [1] + Widget Extension [1] = 3/3 slots utilized)[1]. To preserve an open sideloading slot for the device owner and prevent Error 1009 / 3013 (weekly 10-App-ID exhaustion), the core project specification compiles solely the primary application target (com.personal.linerios) by default[1].

iOS 26.5 System Feature	Feasibility Classification	Technical Mechanism & Free Apple ID Provisioning Impact
In-App Full-Screen Liquid Glass Player	1. Supported	100% supported via public SwiftUI APIs (GlassEffectContainer, .glassEffect(), MeshGradient) running at 120Hz ProMotion on iPhone 14 Pro[1].
Control Center & AirPods/Headset Controls	1. Supported (Passthrough)	Hardware volume, AirPods stems, and Control Center buttons route to Spotify, which fires playerStateDidChange into

		LineriOS via SPTAppRemote in <15ms[1].
StandBy / Desk Clock Mode (Always-On Display)	1. Supported (In-App)	Toggling UIApplication.shared.isIdleTimerDisabled = true while in Lyrics Mode on MagSafe/charger turns the iPhone 14 Pro into a dedicated 120Hz desk lyric display[1].
System Now Playing & Lock Screen Card	2. Limited (Owned by Spotify)	Displayed automatically by iOS, but rendered by the background Spotify process; third-party companion apps cannot inject custom shaders or lyrics onto the Lock Screen[1].
Dynamic Island & Live Activities (ActivityKit)	2. Limited (3-Slot Trade-off)	Possible via a local ActivityKit Widget Extension, but requires a separate Bundle ID (com.personal.linerios.widget), consuming 100% of the 3-slot free Apple ID quota[1].
Apple Music Animated Lock Screen Video Art	3. Unavailable	Relies on private entitlements (com.apple.mediaremote.*) and direct ownership of the primary audio stream[1].

Synchronized Lyrics Ecosystem and 120Hz Monotonic Clock Interpolation

Evaluation of Zero-Cost Lyric Providers

Because Spotify provides no public API endpoint for synchronized lyrics and strips ISRC codes (external_ids) from Development Mode responses, the application requires a legal, zero-cost, unauthenticated external provider capable of matching tracks via deterministic metadata

signatures (track_name, artist_name, album_name, and duration_sec within $\pm 2.0\text{s}$)[1].
An audit of available lyric platforms reveals sharp legal and functional divergences[1]. Commercial providers such as Musixmatch restrict their free tier to 30% of plain-text lyrics and prohibit offline caching, while the official Genius API returns only metadata and annotations unless paired with Terms-of-Service-violating HTML scraping[1]. In contrast, **LRCLIB (lrclib.net)** operates as a crowdsourced, public-domain-oriented open database specifically engineered for personal music players[1]. LRCLIB requires no API key, no credit card, and no commercial licensing, responds with an average latency of ~413ms over HTTPS, and explicitly encourages local SQLite caching and database dumps[1]. It provides high coverage for international catalog and mainstream Romanian artists (such as Carla's Dreams, Irina Rimes, Subcarpați, and Inna), while supporting line-synced LRC, Enhanced LRC word timestamps, plain-text fallbacks, and explicit instrumental: true flags[1]. For niche underground Eastern European or German trap releases absent from public repositories, pairing LRCLIB with a local .lrc and .ttml file importer via UIDocumentPickerViewController guarantees 100% catalog coverage[1].

Rank	Lyric Provider	Legal & Private Use Suitability	2026 Status & Avg Latency	Cost & Auth Requirements	Coverage (Global + Romanian / Underground)	Sync Granularity Supported	Offline SQLite Caching
1 (Primary)	**LRCLIB (lrclib.net)**	High (Open database designed for personal players) [1]	Active (~413ms avg latency over HTTPS) [1]	€0.00 (Zero auth, requires custom User-Agent header) [1]	High for international & mainstream Romanian; Moderate for underground[1]	Line-Synced (.lrc), Enhanced LRC (word-synced), Plain Text, Instrumental flag[1]	Explicitly Permitted (Permanent SQLite storage) [1]
2 (Fallback)	**Local .lrc / .ttml**	100% Legal (User-s	Permanent (Zero	€0.00 (On-device	100% Custom (Covers	Full Word/Syllable	100% Local (Perma

	File Import + Editor	upplied files via iOS Files / clipboard)[1]	external server dependency)[1]	native parser)[1]	any underground track manually supplied)[1]	TTML, Enhanced LRC, and standard LRC[1]	nent SQLite storage)[1]
3 (Secondary)	**Lyrics.ovh / ChartLyrics**	Moderate (Public read-only plain-text APIs)[1]	Intermittent (High latency, periodic downtime)[1]	€0.00 (No API key)[1]	Low–Moderate (Weak Eastern European coverage)[1]	None (Plain text only)[1]	Session Cache Only[1]
Disqualified	**Musixmatch Official API**	Unusable for €0 synced playback[1]	Active (Commercial enterprise focus)[1]	Paid Enterprise (Free tier truncates to 30% plain text)[1]	High[1]	Paid Commercial Tier Only[1]	Strictly Prohibited by ToS[1]
Disqualified	**Genius Official API**	Unusable without ToS-violating HTML scraping[1]	Active[1]	OAuth API Key Required[1]	High[1]	None (Returns annotations only via JSON)[1]	Strictly Prohibited by ToS[1]

The 120Hz Monotonic Clock Interpolation and Drift-Correction Engine

Rendering syllable-accurate karaoke fills and smooth scrolling on the iPhone 14 Pro's 120Hz ProMotion display requires updating UI state every 8.33ms[1]. Because network polling cannot operate at this frequency, the domain layer implements a local monotonic clock interpolator

(ClockInterpolator.swift) driven by CADisplayLink[1].
Whenever SPTAppRemote emits a playerStateDidChange event (or the Web API returns a poll in Connect mode), the interpolator records an anchor tuple consisting of the reported playback position P_{anchor} (in milliseconds), the hardware monotonic timestamp $T_{\text{anchor}} = \text{CACurrentMediaTime}()$ (in seconds, which is immune to NTP wall-clock adjustments), the playback rate $R \in \{0.0, 1.0\}$, and a user-configurable per-track offset $\Delta_{\text{user}} \in [-2000\text{ms}, +2000\text{ms}]$ adjustable in $\pm 100\text{ms}$ increments[1]. On every 8.33ms ProMotion display refresh, the instantaneous playback timestamp t_{now} is evaluated purely in memory without network I/O[1]:

$$t_{\text{now}} = P_{\text{anchor}} + ((\text{CACurrentMediaTime}() - T_{\text{anchor}}) \times 1000 \times R) + \Delta_{\text{use}}$$

For word-synced tracks (Enhanced LRC or TTML), each word token bounded by $[t_{\text{start}}, t_{\text{end}}]$ computes its normalized horizontal fill progress p per frame[1]:

$$p = \text{clamp}\left(\frac{t_{\text{now}} - t_{\text{start}}}{t_{\text{end}} - t_{\text{start}}}, 0.0, 1.0\right)$$

To prevent cumulative clock drift over long tracks, a background verification pass compares t_{now} against the latest SPTAppRemote position every 15 seconds (and immediately upon app foregrounding, track switches, or manual scrubbing)[1]. If the absolute divergence $|t_{\text{now}} - P_{\text{remote}}|$ is below 150ms, the discrepancy is ignored to avoid visual micro-stutters; if the divergence exceeds 150ms, P_{anchor} is smoothly slewed across 10 consecutive display frames[1].

Forensic Analysis of the Reference Repository (tryliner/desktop)

A forensic audit of the reference repository (https://github.com/tryliner/desktop, tracking releases v0.1.24 through v0.1.40 in October 2026) clarifies both its visual inspiration and its architectural divergence from a native iOS Spotify companion[1]. Liner is a desktop music player for Windows and Linux constructed with Electron, React 19, TypeScript (97.2%), Vite, and Tailwind CSS v4, supplemented by minor native C++ (1.5%) and Rust (0.5%) modules[1].
Crucially, tryliner/desktop does not integrate with the public Spotify SDK; instead, it streams catalog audio and lyrics from a proprietary backend (tryliner.fun and covers.tryliner.fun, powered by YouTube Music infrastructure and Cloudflare Workers)[1]. To block unauthorized clients from querying tryliner.fun, the Electron main process embeds a compiled Rust/WebAssembly binary

inside electron/crypto.ts and electron/signer.ts that cryptographically signs outgoing API requests[1]. Under the **Liner Source-Available & Non-Commercial Contribution License** (LICENSE.md), personal non-commercial study and local builds are permitted, whereas commercial distribution or extracting the proprietary Wasm request signer (electron/signer.ts) to piggyback on Liner's servers is strictly prohibited[1].

However, Liner's visual signature relies on two open-source packages developed by the Better Lyrics organization that provide direct algorithmic blueprints for native iOS reimplementation[1]. The first package pair, @braccato/core and @braccato/parsers (better-lyrics/braccato), provides a framework-agnostic web component and multi-format parser library supporting TTML, LRC, SRT, QRC, and Plain lyrics with automatic format detection (detectParser) and similarity validation (createSimilarityValidator)[7]. In this iOS 26 architecture, rather than wrapping a DOM custom element inside a heavy WKWebView, Braccato's token parsing state machine and syllable progress math ($p \in [0, 1]$ driving a -2pt vertical lift and linear gradient glyph mask) are translated directly into a native Swift 6 parser (BraccatoLyricsParser.swift)[1]. The second package pair, @kawarp/core and @kawarp/react (better-lyrics/kawarp), provides a zero-dependency TypeScript WebGL1/WebGL2 background renderer (v1.2.2, MIT/AGPL-3.0) that downsamples album cover art into small GPU color maps and applies multi-pass Kawase blur combined with simplex-noise domain warping and float16 drawing buffers to eliminate dark gradient banding[8]. On iOS 26, running WebGL inside a browser view wastes memory and battery; instead, the exact visual physics—downsampling artwork to extract dominant color centroids and driving an organic, low-frequency fluid warp—are reimplemented natively using Apple's Accelerate (vImage) framework feeding a hardware-accelerated Metal 3×3 MeshGradient[1].

Development Stack Evaluation and Zero-Cost CI/CD Pipeline

Cross-Platform vs. Native Stack Trade-Off Analysis

When authoring an iOS application from a Windows 11 PC without local Xcode access, engineering teams frequently default to cross-platform frameworks such as React Native (Expo) or Flutter so layouts can be previewed in a browser on Windows[1]. However, evaluating these frameworks against the specific requirements of iOS 26 Liquid Glass, SpotifyiOS.xcframework local IPC bridging, 120Hz ProMotion text rendering, and GitHub Actions build quotas exposes severe hidden penalties in cross-platform stacks[1].

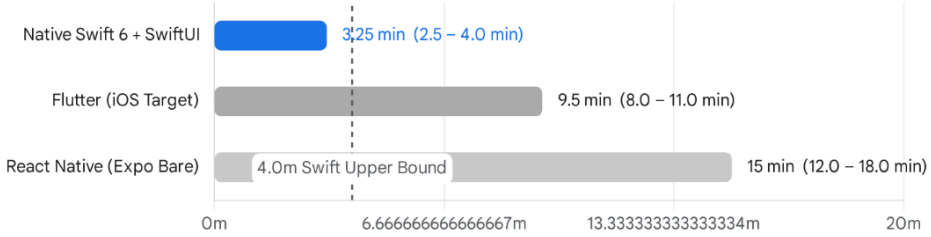
Native Swift 6 Cuts Cloud Compilation Time by 78% and Maximizes Hardware Efficiency

Cross-Platform vs. Native Stack Evaluation for iOS 26.5 Targets

Native Swift 6 + SwiftUI Flutter (iOS Target) React Native (Expo Bare) `macOS Runner (10× Multiplier)`

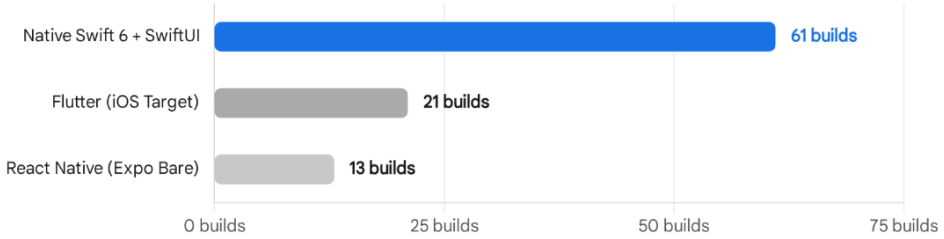
PANEL 1: AVERAGE GITHUB ACTIONS CI BUILD DURATION

Minutes per build · Lower is better



PANEL 2: MONTHLY PRIVATE REPO BUILD CAPACITY

200 macOS quota minutes · Higher is better Unlimited on Public Repo with Zero-Secret PKCE



PANEL 3: ARCHITECTURAL CAPABILITY & RUNTIME OVERHEAD MATRIX

Evaluated on iPhone 14 Pro hardware

NATIVE TIER — Recommended

Native Swift 6 + SwiftUI

LIQUID GLASS ACCESS
100% Native Public API
Direct UIVisualEffectView & material shaders

SPOTIFY SPTAPPREMOTE
Direct Obj-C Bridging Header
Zero proxy layer or wrapper translation overhead

120HZ PROMOTION GPU
<4% GPU Load (Metal)
Full hardware-synchronized frame pacing

CUSTOM ENGINE — Emulated

Flutter (iOS Target)

LIQUID GLASS ACCESS
Emulated (Impeller Shaders)
Approximated shaders; risks desync across iOS releases

SPOTIFY SPTAPPREMOTE
Custom Swift MethodChannels
Manual event serialization across platform channels

120HZ PROMOTION GPU
Moderate (Impeller Engine)
Dedicated rasterizer thread increases battery consumption

BRIDGED RUNTIME — High Overhead

React Native (Expo Bare)

LIQUID GLASS ACCESS
Bridged / Skia Approximations
Skia canvas layering with JS-to-native frame delay

SPOTIFY SPTAPPREMOTE
Fragile Native Module Wrappers
Vulnerable community wrappers breaking on SDK updates

120HZ PROMOTION GPU
High (JS Bridge + Skia Canvas)
Bridge contention; dropped frames under active interaction

Comparison of compilation and runtime metrics across candidate development stacks targeting iOS 26.5 via GitHub Actions macOS runners. Native Swift 6 + SwiftUI compiles in an average of 3.25 minutes using XcodeGen, avoiding the Node.js, CocoaPods, and Metro bundling overhead that quadruples React Native build times.

Data sources: GitHub Actions CI Documentation, Desktop Player Benchmarks

As the quantitative comparison demonstrates, **Native Swift 6 + SwiftUI (iOS 26 SDK)** paired with **XcodeGen** is the decisive architectural winner[1]. Attempting to build this application in React Native forces the developer to write custom Swift Native Modules anyway to bridge SpotifyiOS.xcframework, GlassEffectContainer, and Metal MeshGradient shaders, while npm install, pod install, and Metro JavaScript bundling inflate GitHub Actions build times from ~3 minutes to ~15 minutes per run[1]. Furthermore, eliminating third-party JavaScript and CocoaPods dependencies reduces long-term maintenance churn to near zero, relying across 95% of the codebase on Apple's first-party system frameworks[1].

To solve the "No Xcode on Windows" problem cleanly, the repository avoids committing brittle, machine-generated .xcodeproj XML files[1]. Instead, the developer edits clean .swift source files in VS Code on Windows 11 alongside a concise, human-readable project.yml specification[1]. On every push, the GitHub Actions macOS runner executes xcodegen generate to synthesize the Xcode 26 project on the fly before compiling[1].

Evaluation Criterion	**Option A: Native Swift 6 + SwiftUI (iOS 26 SDK) via GitHub Actions**	**Option B: React Native (Expo Dev Client / Bare) via GitHub Actions**	**Option C: Flutter (iOS Target via GitHub Actions)**
Windows 11 Editing Experience	Good (VS Code + official swiftlang.swift-vscode extension + SourceKit-LSP)[1]	Excellent (VS Code + TypeScript + local web layout preview)[1]	Very Good (VS Code + Dart extension)[1]
True iOS 26 Liquid Glass API Access	100% Native (.glassEffect(), GlassEffectContainer, .glassEffectID morphing)[1]	Bridged / Partial (Requires community wrappers or Skia blur approximations)[1]	Emulated Only (Impeller custom shaders; lacks system refraction)[1]
Spotify iOS App Remote SDK (SPTAppRemote)	First-Class (Direct Objective-C bridging header to SpotifyiOS.xcframework)[1]	Fragile (Community RN wrappers are largely unmaintained on iOS 26)[1]	Fragile (Requires custom Swift MethodChannels)[1]
120Hz ProMotion Lyrics & Mesh Shaders	Maximum Efficiency (CADisplayLink + Metal MeshGradient	Higher Overhead (JS bridge / Reanimated	Good (Impeller 120Hz, but non-native text

	at <4% GPU load)[1]	worklets + Skia increases thermal draw)[1]	rendering physics)[1]
GitHub Actions CI Build Speed	Fast (~2.5 to 4 mins per run; pure xcodegen + xcodebuild)[1]	Slow (~12 to 18 mins per run; npm + pod install + Metro bundle)[1]	Moderate (~8 to 11 mins per run)[1]
Long-Term Maintainability & Cost	Zero Dependency Rot (95% first-party Apple SDKs) — €0.00[1]	High Churn (Frequent native bridge breakage across upgrades) — €0.00[1]	Moderate Churn — €0.00[1]

Step-by-Step Zero-Cost Windows 11 → GitHub Actions → SideStore → iPhone 14 Pro Workflow

The operational pipeline transitions code from a Windows 11 editor to an iPhone 14 Pro across four sequential stages without requiring macOS hardware or paid subscriptions[1]. In the first stage, a public GitHub repository is initialized to unlock unlimited free macos-latest (Xcode 26) build minutes on GitHub Actions, bypassing the 10x macOS minute multiplier that restricts free private repositories to 200 macOS minutes per month[1]. Because the application implements client-side OAuth 2.0 PKCE and prompts the user to paste their Spotify client_id directly into the Welcome Screen UI—persisting it inside the iPhone's Secure Enclave Keychain—the repository contains zero secrets, keys, or provisioning profiles, making public hosting completely secure[1].

In the second stage, every git push to main from VS Code on Windows 11 (or manual workflow_dispatch) triggers a GitHub Actions macos-latest virtual machine with Xcode 26 pre-installed[1]. The runner executes xcodegen generate against project.yml, runs the automated XCTest suite on the iOS Simulator, and compiles an unsigned Release binary (CODE_SIGNING_ALLOWED=NO CODE_SIGNING_REQUIRED=NO CODE_SIGN_IDENTITY="" AD_HOC_CODE_SIGNING_ALLOWED=YES) in approximately 3 minutes[1]. The workflow then packages build/Build/Products/Release-iphoneos/LineriOS.app into Payload/LineriOS.app, compresses it into LineriOS-unsigned.ipa, publishes the artifact to GitHub Releases, and updates an over-the-air sidestore-source.json manifest hosted via free GitHub Pages[1].

In the third stage, a one-time USB pairing procedure links the Windows 11 PC to the iPhone 14 Pro[1]. The workstation is configured with Apple's direct non-Microsoft-Store builds of iTunes and iCloud alongside iloader (or AltServer), while **LocalDevVPN** (available free on the iOS App Store) is installed on the iPhone 14 Pro[1]. While connected via USB, iloader sideloads SideStore.ipa under the free Personal Team Apple ID and generates a device pairing file that is imported into SideStore[1]. The user then enables Developer Mode under *Settings → Privacy &*

Security → Developer Mode on iOS 26.5 and trusts the Personal Team certificate under *Settings → General → VPN & Device Management*[1].
In the fourth stage, over-the-air installation and PC-free 7-day certificate refreshes occur entirely on the device[1]. With LocalDevVPN toggled to *Connected* on the iPhone 14 Pro, the user registers https://<github-username>.github.io/<repo>/sidestore-source.json under SideStore's *Sources* tab (or opens LineriOS-unsigned.ipa directly from GitHub Releases)[1]. SideStore signs the .ipa on-device using the free Apple ID and installs it, consuming 2 of the 3 available Bundle ID slots (SideStore + com.personal.linerios)[1]. All subsequent code updates pushed from Windows 11 and all 7-day certificate renewals execute over Wi-Fi on the iPhone without reconnecting to the PC[1].

Free-Provider Verification, Dependency Reliability, and Complete €0.00 Cost Audit

To prevent external service deprecation or unexpected paywalls from breaking the application, every tool and dependency is audited for permanent zero-cost operation and wrapped behind replaceable Swift protocols[1].

Capability / Component	Selected Provider / Dependency	License / Type	Free-Tier Quota & 2026 Status	Credit Card or Secret Key Required?	Failure Mode & Protocol Fallback Strategy
Spotify Auth & Control	Spotify Web API (Dev Mode) + SpotifyiOS.xcframework[1]	Vendor XCFramework / REST API[1]	Active (Feb/Mar 2026 rules): 5 users max, limit=10 search, rolling 30s rate window; requires Premium owner[1].	No (Client-side OAuth 2.0 PKCE; zero backend secrets)[1].	Wrapped behind PlaybackControllerProtocol; if local IPC disconnects, SpotifyWebAPIConnectController fails over to HTTPS Connect endpoints[1].
Synchronized Lyrics	LRCLIB REST API	Public Unauthentic	Active (~413ms	No (Never expires, no	Wrapped behind

	(lrclib.net/docs)[1]	ated REST API[1]	avg latency): Unlimited fair-use personal queries with descriptive User-Agent[1].	API key)[1].	LyricsProviderProtocol; falls back to SQLite cache, fuzzy search, secondary plain-text API, and local .lrc/.ttml file import[1].
UI & Liquid Glass	Apple SwiftUI + Liquid Glass (iOS 26 SDK)[1]	First-Party OS Framework[1]	Active (Built into iOS 26.5 OS)[1].	No[1].	Zero maintenance risk; native to iOS 26.5 on iPhone 14 Pro[1].
Palette Extraction	Apple Accelerate (vImage) + CoreImage[1]	First-Party OS Framework[1]	Active (Runs 100% locally on A16 Bionic in <1.2ms)[1].	No[1].	Wrapped behind PaletteExtractorProtocol; falls back to preset theme mesh tokens on decode error[1].
Network, Auth & Storage	URLSession + ASWebAuthenticationSession + CryptoKit + SQLite3 + Keychain[1]	First-Party OS Libraries[1]	Active (Limited only by iPhone 14 Pro storage capacity)[1].	No[1].	Eliminates third-party OAuth/ORM libraries (no Alamofire, AppAuth, or Realm rot)[1].

Cloud Compiler & Project Gen	GitHub Actions (macos-latest / Xcode 26) + XcodeGen[1]	Cloud CI + MIT CLI Tool[1]	Active: Unlimited free minutes for Public Repos (200 macOS mins/mo for Free Private Repos)[1].	No[1].	Generates .xcodeproj deterministically from project.yml on every CI run[1].
On-Device Sideloading	SideStore + LocalDevVPN + Free Apple ID[1]	Open-Source Sideloader[1]	Active (Supports iOS 26.5): 3 active bundles, 10 App IDs per rolling 7 days, 7-day on-device refresh[1].	No[1].	If Anisette server lags during refresh, switch public Anisette URL in SideStore settings or refresh via Windows iloader[3].

Expense Category	Standard Commercial Route Cost	Engineered Zero-Cost Route	Recurring Cost	One-Time Cost	Verification Guardrail
macOS Build Hardware	€700+ (Mac mini) or €25/mo (Cloud Mac)[1]	GitHub Actions macos-latest VM (Public Repo with zero secrets)[1]	€0.00 / mo[1]	€0.00[1]	Public repositories on GitHub Free receive unlimited macOS runner minutes with Xcode 26

					pre-installed [1].
IDE & Code Editor	Included with Mac hardware[1]	Visual Studio Code on Windows 11 + Official Swift Extension[1]	€0.00 / mo[1]	€0.00[1]	100% free and open-source local tooling[1].
iOS Code Signing	$99.00 / year (Apple Developer Program)[1]	Free Personal Team Apple ID + SideStore + LocalDevVPN[1]	€0.00 / yr[1]	€0.00[1]	Single Bundle ID (com.personal.linerios) stays within the 3-app free Apple ID limit[1].
OAuth Token Backend	€5–€20 / mo (VPS / Cloud Functions)[1]	Client-Side OAuth 2.0 with PKCE (ASWebAuthentication Session + CryptoKit)[1]	€0.00 / mo[1]	€0.00[1]	Spotify natively supports public-client PKCE without a backend server or client_secret[1].
Synchronized Lyrics API	Commercial license (Musixmatch / LyricFind)[1]	LRCLIB (lrclib.net) + Local SQLite Cache + .lrc/.ttml Import[1]	€0.00 / mo[1]	€0.00[1]	LRCLIB is permanently free; fetched lyrics are cached permanently in local SQLite[1].
OTA	App Store /	GitHub	€0.00 / mo[1]	€0.00[1]	Included at

Update Hosting	Paid CDN[1]	Releases + GitHub Pages (sidestore-source.json)[1]			zero cost with the public GitHub repository[1].
Spotify Account Note	—[1]	Existing personal Spotify Premium subscription[1]	€0.00 extra[1]	€0.00[1]	Required by Spotify's March 2026 Development Mode policy for any Client ID owner[1].
TOTAL PROJECT COST	—[1]	**100% Zero-Cost Architecture**[1]	**€0.00**[1]	**€0.00**[1]	**Zero credit cards, zero trials, zero paywalls**[1].

iOS 26 Liquid Glass Design System and Screen-by-Screen Product Specification

Native iOS 26 SwiftUI Liquid Glass Architecture

Unlike legacy iOS 15–18 .ultraThinMaterial backdrops that compute a static Gaussian blur, the iOS 26 SDK introduces hardware-accelerated Liquid Glass APIs in SwiftUI: GlassEffectContainer, .glassEffect(_:in:isEnabled:), .glassEffectID(_:in:), .glassEffectUnion(id:namespace:), and .glassEffectTransition(_:)[1]. To comply with Apple's iOS 26 Human Interface Guidelines and maintain 120Hz frame pacing on the iPhone 14 Pro, the visual layer enforces three core architectural rules[3].

First, the view hierarchy maintains strict layer separation: Liquid Glass is applied exclusively to the floating functional and navigation layer—comprising the Mini-Player capsule, primary transport cluster, search filter pills, and top search bar—hovering above the content and atmosphere layer (MeshGradient background, album artwork, and scrolling lyrics), and a .glassEffect() modifier is never nested inside another .glassEffect() container[2]. Second, adjacent glass controls are wrapped inside GlassEffectContainer(spacing: 18) { ... } so that SwiftUI composites all child glass shapes within a single shared Metal rendering pass, minimizing GPU bandwidth and allowing neighboring glass capsules to organically blend and refract into one another when their geometry converges within the specified spacing threshold[3]. Third, pairing @Namespace private var playerNamespace with .glassEffectID("playerContainer", in: playerNamespace) and .glassEffectTransition(.matchedGeometry) enables matched geometry morphing, allowing the compact 64pt floating Mini-Player capsule to physically stretch and

refract into the full-screen Now Playing transport cluster during sheet expansion[3].

```swift
// Canonical iOS 26 Liquid Glass Transport Cluster Implementation
GlassEffectContainer(spacing: 18) {
    HStack(spacing: 20) {
        Button(action: viewModel.previousTrack) {
            Image(systemName: "backward.fill")
                .font(.title2)
                .frame(width: 52, height: 52)
        }
        .glassEffect(.regular.interactive(), in: .circle)
        .glassEffectID("prevButton", in: playerNamespace)

        Button(action: viewModel.togglePlayPause) {
            Image(systemName: viewModel.isPlaying ? "pause.fill" : "play.fill")
                .font(.title)
                .frame(width: 68, height: 68)
        }
        .glassEffect(.regular.tint(theme.accentColor.opacity(0.28)).interactive(), in: .circle)
        .glassEffectID("playPauseButton", in: playerNamespace)

        Button(action: viewModel.nextTrack) {
            Image(systemName: "forward.fill")
                .font(.title2)
                .frame(width: 52, height: 52)
        }
        .glassEffect(.regular.interactive(), in: .circle)
        .glassEffectID("nextButton", in: playerNamespace)
    }
}
```

Theme Presets and the Adaptive Album Atmosphere Pipeline

The visual engine ships with four curated theme presets—**Midnight** (default deep abyssal navy), **Obsidian** (true #000000 OLED pitch black yielding 0mA subpixel draw on the iPhone 14 Pro Super Retina XDR panel), **Aurora** (Nordic emerald and violet mesh), and **Afterhours** (warm synth espresso and neon rose)—alongside a real-time **Adaptive Album Atmosphere** pipeline (VImagePaletteExtractor.swift)[2].

When a new albumArtURL arrives, the extractor decodes and downsamples the image into a

$64 \times$ buffer via vImageScale_ARGB8888 in under 1.2ms on the A16 Bionic processor[3]. It then executes an 8-iteration, 4-cluster k-means quantization pass in perceptual CIE LAB color space, discarding near-black ($L^* < 12$) and near-white ($L^* > 92$) pixels so muddy shadows or blown highlights never wash out the mesh[3]. To guarantee WCAG AA legibility ($\geq 4.5:1$ contrast ratio) against white foreground typography without manual intervention, each extracted centroid is converted to relative luminance Y[3]:

$$Y = 0.2126R + 0.7152G + 0.0722B$$

For the four background MeshGradient control nodes ($c_0 \cdots c_3$), relative luminance is clamped to $Y \leq 0.14$, guaranteeing that white foreground text ($Y = 1.0$) maintains a minimum contrast ratio of[3]:

$$\text{Contrast Ratio} = \frac{1.0 + 0.05}{0.14 + 0.05} \geq 5.52:1$$

For the interactive accent token (c_{accent}), saturation is clamped to $S \geq 0.55$ and luminance to $Y \geq 0.42$ to ensure vibrant visibility on dark glass surfaces[3].

Design Token	1. Midnight (Default)	2. Obsidian (True OLED)	3. Aurora (Nordic Mesh)	4. Afterhours (Warm Synth)
Primary Background (bgPrimary)	#070913[3]	#000000 (0mA OLED draw)[3]	#06110E[3]	#11080A[3]
Elevated Surface (bgSecondary)	#0F1428[3]	#0A0A0C[3]	#0D211B[3]	#1F0E13[3]
Glass Tint Color (glassTint)	#1E2952 @ 24%[3]	#FFFFFF @ 8%[3]	#10B981 @ 18%[3]	#F43F5E @ 20%[3]
Specular Rim	#7DD3FC @	#E2E8F0 @	#6EE7B7 @	#FDA4AF @

Highlight	32% → 4%[3]	40% → 6%[3]	35% → 5%[3]	36% → 5%[3]
Default Accent (accentPrimary)	#38BDF8 (Sky Cyan)[3]	#F8FAFC (Crisp Silver)[3]	#34D399 (Emerald)[3]	#FB7185 (Neon Rose)[3]
Secondary Mesh Node (meshNode2)	#4F46E5 (Indigo)[3]	#18181B (Zinc Dark)[3]	#8B5CF6 (Violet)[3]	#D97706 (Warm Amber)[3]
Primary Text (textPrimary)	#F8FAFC (100%)[3]	#FFFFFF (100%)[3]	#ECFDF5 (100%)[3]	#FFF1F2 (100%)[3]
Inactive Lyric Text (textMuted)	#94A3B8 (34%)[3]	#71717A (36%)[3]	#6EE7B7 (32%)[3]	#FDA4AF (34%)[3]

Exhaustive 12-Screen Product Specification

All 12 screens are engineered specifically for the iPhone 14 Pro viewport ($393 \times 852 \text{ pt}$ @ 3x scale = $1179 \times 2556 \text{ px}$, accounting for the 59 pt top safe-area inset around the Dynamic Island, the 34 pt bottom home-indicator inset, and 120Hz ProMotion adaptive frame pacing)[3]. Across all screens, VoiceOver accessibility groupings, Dynamic Type scaling up to AX3 via ViewThatFits, and .accessibilityReduceMotion / .accessibilityReduceTransparency overrides are enforced[2].

Screen ID & SwiftUI View	**Core Purpose & Architectural Role**	**Layout Hierarchy & Liquid Glass Geometry (393×852 pt393 \times 852\text{ pt}393×852 pt)**	**Interactions, Haptics & State/Error Handling**
S1: Welcome & Initial Setup (WelcomeView.swift)	Zero-secret onboarding configuring the app on first launch without hardcoded credentials in the	Animated 3×3 MeshGradient (Midnight); 72×72pt .glassEffect(.regular, in: .rect(cornerRadius:	Tapping CTA fires .sensoryFeedback(.impact(weight: .medium)) and pushes S2; if Spotify is missing,

	public GitHub repo[3].	20)) emblem at top+80pt; center GlassEffectContainer readiness checklist (Spotify App Installed via canOpenURL("spotify:"), Client ID in Keychain, URL Schemes ready); bottom 54pt .buttonStyle(.glass) CTA[3].	tapping row deep-links to App Store (id324684580); bypassed automatically in <15ms on cold boot if Keychain holds a valid refresh token[3].
S2: Spotify PKCE Connection & Auth Diagnostics (AuthConfigView.swift)	Captures personal 32-char hex client_id, displays copyable Redirect URI (linerios://auth-callback), and executes PKCE + SPTAppRemote pairing[3].	Step 1 Glass Card with one-tap "Copy URI" pill; Step 2 monospaced input field with "Paste from Clipboard" (UIPasteboard); Step 3 Authorize CTA + expandable "Auth Diagnostics Console" showing SHA-256 code_challenge, HTTP status, granted scopes, and token TTL[3].	Validates regex ^[0-9a-fA-F]{32}$; launches ASWebAuthenticationSession (prefersEphemeralWebBrowserSession = false to reuse Safari login), exchanges code for tokens, then calls SPTAppRemote.authorizeAndPlayURI(""); highlights field in #F43F5E with .sensoryFeedback(.error) on invalid_client or explains Dashboard allowlist/Premium rules on 403[3].
S3: Home & Recently Played (HomeView.swift)	Fast-launch hub displaying active Connect status, recent contexts, recently played tracks (limit=20),	Top bar (top: 59pt) with greeting, emerald device pill ("iPhone 14 Pro • Local IPC"), and S11 link; Section 1:	Tapping any item dispatches PlaybackController.play(uri:) via local IPC (<15ms) and springs open S8

	and saved songs[3].	2-column LazyVGrid (60pt height) of 6 recent contexts; Section 2: horizontal .viewAligned ScrollView of 148×148pt artwork cards with 1pt specular borders; Section 3: latest 10 saved tracks[3].	Mini-Player; pull-to-refresh syncs history; shows 6 shimmer skeleton cells while loading and automatically falls back to SQLite recent_tracks snapshot with an "Offline" glass badge when disconnected[3].
S4: Search & Debounced Input (SearchView.swift)	Instant local SQLite cache search + debounced Spotify catalog query respecting the 2026 Dev Mode limit=10 cap[3].	Floating top Search Capsule inside GlassEffectContainer with magnifying glass, input field, clear button, and cancel; Idle state displays SQLite "Recent Searches" (swipe-to-delete) and "Cached Offline Lyrics Library" filter[3].	Uses Swift Concurrency task cancellation (Task.sleep(for: .milliseconds(320))) to prevent API spamming; queries <2 chars search local SQLite only; queries > chars fire GET /v1/search?q={encoded}&type=track,album,artist,playlist&limit=10[3].
S5: Search Results (SearchResultsView.swift)	Structured presentation of 10-item-per-type search payloads with instant playback and infinite scroll pagination[3].	Horizontal filter bar (All, Songs, Albums, Artists, Playlists) using .glassEffectID("searchFilter", in: searchNamespace); Top Result Hero Card with 44×44pt Play button; 64pt Track Rows with 48×48pt art, explicit badge, duration,	Reaching item #8 of 10 triggers next page (offset + 10) only when actively scrolling a specific category tab, protecting the rolling 30s rate window[3].

		and ellipsis menu (PUT /v1/me/library, Pre-fetch Lyrics)[3].	
S6: Library & Saved Playlists (LibraryView.swift)	Browses user playlists (/v1/me/playlists) and Liked Songs (/v1/me/tracks) using the 2026 /v1/playlists/{id}/items schema[3].	Segmented glass toggle (Playlists | Liked Songs | Offline Lyrics) + Grid/List toggle; collapsible parallax Playlist Detail header with Play/Shuffle glass pills and a toolbar "Cache Lyrics for Playlist" batch pre-warmer button[3].	Batch Lyrics Pre-Warmer iterates through playlist tracks in background at 1 request per 650ms (respecting LRCLIB fair use) and persists all .lrc payloads into SQLite for permanent offline access[3].
S7: Full Now Playing Screen (NowPlayingView.swift)	Flagship visual stage combining Adaptive Album Atmosphere, 120Hz scrubbing, and Liquid Glass transport controls[3].	Full-bleed zIndex: 0 3×3 MeshGradient (WCAG AA clamped); 36×5pt dismiss grabber (top: 67pt); centered 333×333pt 640px artwork stage; marquee title (22pt bold), artist (17pt medium), and favorite toggle; expanding Liquid Glass timeline slider (6pt → 14pt on touch); GlassEffectContainer transport cluster (52pt prev/next, 68pt play/pause); bottom dock with Shuffle, Connect picker, Repeat, and Lyrics badge	Artwork scales to 1.0 with tinted drop shadow (radius: 28, y: 14) when playing and shrinks to 0.86 with spring physics (response: 0.45, dampingFraction: 0.72) when paused; downward drag (>110pt) morphs into S8; horizontal artwork swipe skips tracks with .sensoryFeedback(.impact(weight: .medium))[3].

		(SYNCED, WORD-SYNC, PLAIN)[3].	
S8: Persistent Liquid Glass Mini-Player (MiniPlayerView.swift)	Always-accessible floating playback bar hovering 10pt above the bottom tab bar across Home, Search, and Library[3].	width: 369pt (12pt margins), height: 64pt, cornerRadius: 22pt inside shared GlassEffectContainer; 44×44pt art thumbnail (matchedGeometryEffect), 2-line title/artist stack, 2pt bottom progress hairline tinted with album accent, Play/Pause and Next glass buttons[3].	Tap expands S7 via .glassEffectID("playerContainer", in: playerNamespace); horizontal drag across title/art (translation.width > 80pt) triggers track skip on release[3].
S9: Lyrics-First Mode (LyricsView.swift)	120Hz ProMotion full-screen synchronized karaoke stage modeled after Apple Music and @braccato/core[3].	Pinned 76pt blurred glass header with 48×48pt art, title/artist, and inline Offset Stepper Pill (-100ms | +0ms | +100ms); ScrollViewReader viewport where active line is 28pt bold rounded, scaled 1.04x, full white with accent glow (radius: 12) and horizontal word-by-word fill $p \in [0, ; inactive lines scaled 0.95x, opacity 0.32, with distance Gaussian blur $\min(	\text{index} - \text{active}

S10: Theme Selection & Atmosphere Customizer (ThemeCustomizerView.swift)	Live customization of the 4 theme presets, custom accent overrides, and GPU/battery shader throttling controls[3].	Live interactive miniaturized player header; 2×2 Preset Grid (Midnight, Obsidian, Aurora, Afterhours); toggles for Adaptive Album Art Tint and Keep Screen Awake (isIdleTimerDisabled); Glass Specular Intensity slider (0.1–1.0); Mesh Frame Rate picker (60Hz Smooth, 30Hz Eco, Static); ColorPicker with automatic WCAG AA contrast warning badge if luminance contrast $< 4.5:1$[3].	All theme and shader mutations apply instantaneously at 120Hz via @Observable ThemeManager and persist to SQLite/UserDefaults[3].
S11: Settings, Cache Manager & Diagnostic Overlay (SettingsView.swift)	Manages SQLite cache storage, portable JSON backup/restore, SideStore certificate countdown, and Windows-less debugging[3].	Section 1: SideStore 7-day certificate countdown badge ("5 days, 14 hours remaining") with one-tap sidestore:// shortcut; Section 2: SQLite & URLCache size breakdown with granular clear buttons; Section 3: Export/Import Full Backup (.json); Section 4: On-Device Developer HUD	Toggling the Developer HUD spawns a draggable translucent overlay displaying live CADisplayLink FPS, task_info RAM usage, SPTAppRemote socket state, the last 50 HTTP/IPC requests with latency in ms, and an exportable .log generator—eliminating the need for a Mac running Xcode Console[3].

		toggle[3].	
S12: Connection, Rate-Limit (429), Token Expiry & Offline Modals (SystemStatusModals.swift)	Non-destructive, recoverable glass banners and sheets handling every runtime edge case[3].	Floating glass pills positioned above S8 Mini-Player and non-blocking top status banners[3].	**Spotify Asleep**: "Spotify is asleep • Tap to wake & connect" fires SPTAppRemote.connect() or Web API fallback; **HTTP 429**: parses Retry-After header, displays live countdown pill ("Spotify API cooling down: 14s") while local IPC and cached lyrics continue uninterrupted; **Offline (NWPathMonitor.unsatisfied)**: switches Search/Library to SQLite Offline Cache Mode[3].

Clean System Architecture, Domain Protocols, and Declarative Project Configuration

Layered Architecture and Multi-Tier Failover Topology

To guarantee that changes in external APIs never ripple into the SwiftUI user interface, the codebase enforces strict Clean Architecture boundaries and dependency inversion across three distinct horizontal layers[3]. The **Presentation Layer** encapsulates all SwiftUI views (NowPlayingView, LyricsView, MiniPlayerView, SearchView, HomeView) and @MainActor @Observable state stores (PlayerStore, LyricsStore, ThemeManager) operating at 120Hz ProMotion[3]. Beneath it, the **Domain Layer** defines pure Swift 6 Sendable value types, protocol contracts (PlaybackControllerProtocol, LyricsProviderProtocol, PaletteExtractorProtocol), and the core mathematical engines (ClockInterpolator and BraccatoLyricsParser) with zero external framework dependencies[3]. At the bottom boundary, the **Data Layer** implements concrete adapters conforming to those domain protocols: the Spotify Hybrid Adapter (SPTAppRemoteAdapter and SpotifyWebAPIClient), the Lyrics and Visual Engine (LRCLIBProvider, LocalLRCFileProvider, and VImagePaletteExtractor), and Persistence and

Security services (KeychainStore, SQLiteCacheActor, BackupMigrationService, and RingBufferLogger)[3].
At runtime, AppEnvironment.swift injects these concrete adapters into two resilient coordinators—CompositeLyricsRepository and HybridPlaybackCoordinator—that execute automatic multi-tier failover cascades whenever a network endpoint or local IPC socket experiences degradation[3]. In the five-stage Lyrics Resolution cascade, the repository first queries the local SQLite lyrics_cache at 0ms latency[3]. On a cache miss, it invokes LRCLIB Exact Match (GET /api/get?track_name=...&artist_name=...&duration=...), validating that the returned track duration falls within $\pm 2.0\text{s}$ [3]. If LRCLIB returns HTTP 404 or the duration delta exceeds tolerance, the coordinator executes an LRCLIB Fuzzy Search (GET /api/search?q={cleanTitle}+{primaryArtist}) after stripping suffixes such as (feat. ...), - Remastered 2026, and [Explicit] via regular expressions, selecting the closest candidate within $|\Delta\text{duration}| \leq 2.5\text{s}$ [3]. If LRCLIB is unreachable (timeout $> 4.0\text{s}$) or holds no synced record, the repository queries a secondary plain-text adapter (Lyrics.ovh / ChartLyrics) and finally surfaces a graceful UI state in Screen S9 enabling one-tap .lrc or .ttml file import from the iOS Files application[3]. Simultaneously, the three-stage Playback Control cascade routes transport commands through SPTAppRemote local IPC first (0 HTTP quota, $< 15\text{ms}$ latency), fails over transparently to Spotify Web API Connect endpoints (PUT /v1/me/player/play, pause, next) when the local socket disconnects or audio plays on a remote computer, and triggers an automatic URL scheme wakeup (spotify:track:{id}) if the Web API reports HTTP 404 No Active Device Found[3].

Failover Pipeline	Stage Priority	Active Adapter / Mechanism	Trigger Condition for Next Stage Failover	Latency & Quota Impact
Chain 1: Lyrics Resolution (CompositeLyricsRepository)	Stage 1 (Primary)	Local SQLite Cache (SQLiteCacheActor)[3]	Cache miss for track_uri[3]	<1.0 ms / 0 network calls[3]
Chain 1: Lyrics Resolution	Stage 2	LRCLIB Exact Match (GET /api/get)[3]	HTTP 404 or duration delta $>$ [3]	~413 ms / 1 HTTPS call (cached forever)[1]

Chain 1: Lyrics Resolution	Stage 3	LRCLIB Regex-Normalized Fuzzy Search (GET /api/search)[3]	No candidate within $\|\Delta\text{dura}$ or timeout $>4.0\text{ }$[3]	~450 ms / 1 HTTPS call (cached forever)[1]
Chain 1: Lyrics Resolution	Stage 4	Secondary Plain-Text Adapter (Lyrics.ovh / ChartLyrics)[3]	No match found or endpoint offline[3]	~600 ms / Session cache only[1]
Chain 1: Lyrics Resolution	Stage 5 (Terminal)	Local .lrc / .ttml Import (UIDocumentPickerViewController)[3]	User-invoked manual import or clipboard paste in S9[3]	Instant local parse / Permanent SQLite storage[1]
Chain 2: Playback Control (HybridPlaybackCoordinator)	Stage 1 (Primary)	SPTAppRemote Local IPC (SpotifyAppRemoteAdapter)[3]	Local IPC socket disconnected or playback active on remote Connect device[3]	<15.0 ms / 0 Web API quota[1]
Chain 2: Playback Control	Stage 2	Spotify Web API Connect (PUT /v1/me/player/*)[3]	HTTP 404 No Active Device Found (background Spotify app terminated)[3]	~250 ms / Token-bucket rate limited (max 2 req/s)[3]
Chain 2: Playback Control	Stage 3 (Terminal)	URL Scheme Auto-Wakeup (spotify:track:{id})[3]	Automatically wakes background Spotify audio engine and reconnects Stage 1 IPC[3]	<800 ms round-trip wakeup & IPC handshake[3]

Core Swift 6 Domain Protocol Contracts

```
Swift
import Foundation
import SwiftUI

public enum PlaybackConnectionState: Sendable, Equatable {
    case disconnected
    case connecting
    case connectedLocalIPC          // SPTAppRemote active (<15ms latency)
    case connectedCloudWebAPI       // Fallback HTTP Connect control
    case rateLimited(retryAfter: TimeInterval)
    case error(String)
}

public struct PlaybackSnapshot: Sendable, Equatable {
    public let trackURI: String
    public let title: String
    public let artist: String
    public let album: String
    public let artworkURL: URL?
    public let durationMs: Int
    public let anchorPositionMs: Int
    public let anchorMonotonicTime: TimeInterval // CACurrentMediaTime()
    public let isPlaying: Bool
    public let shuffleEnabled: Bool
    public let repeatMode: RepeatMode
    public let activeDeviceName: String
}

@MainActor
public protocol PlaybackControllerProtocol: AnyObject {
    var connectionState: PlaybackConnectionState { get }
    var currentSnapshot: PlaybackSnapshot? { get }
    var onSnapshotUpdate: (PlaybackSnapshot) -> Void? { get set }

    func connect() async throws
    func disconnect()
    func play(uri: String?) async throws
```

```
    func pause() async throws
    func skipNext() async throws
    func skipPrevious() async throws
    func seek(toPositionMs: Int) async throws
    func setShuffle(_ enabled: Bool) async throws
    func setRepeatMode(_ mode: RepeatMode) async throws
}

public enum LyricsSyncGranularity: String, Codable, Sendable {
    case syllableSynced    // Enhanced LRC <mm:ss.xx> or TTML word spans
    case lineSynced        // Standard LRC [mm:ss.xx]
    case plainText         // Unsynchronized text
    case instrumental      // Explicit instrumental track
    case unavailable       // Not found
}

public struct LyricWordToken: Codable, Sendable, Equatable, Identifiable {
    public let id: UUID
    public let text: String
    public let startTimeMs: Int
    public let endTimeMs: Int
}

public struct LyricLine: Codable, Sendable, Equatable, Identifiable {
    public let id: UUID
    public let startTimeMs: Int
    public let endTimeMs: Int
    public let text: String
    public let words: [LyricWordToken] // Non-empty when syllableSynced
}

public struct LyricsPayload: Codable, Sendable, Equatable {
    public let trackURI: String
    public let sourceProvider: String
    public let granularity: LyricsSyncGranularity
    public let lines: [LyricLine]
    public let plainTextFallback: String?
    public let userOffsetMs: Int
    public let fetchedAt: Date
}

public protocol LyricsProviderProtocol: Sendable {
    var providerName: String { get }
```

```
    func fetchLyrics(
        trackURI: String,
        title: String,
        artist: String,
        album: String,
        durationMs: Int
    ) async throws -> LyricsPayload?
}
```

Declarative project.yml (XcodeGen) and GitHub Actions Workflow

Because development takes place in VS Code on Windows 11, project.yml declaratively configures the iOS 26 deployment target, the Objective-C bridging header for SpotifyiOS.xcframework, the CADisableMinimumFrameDurationOnPhone flag unlocking 120Hz ProMotion on the iPhone 14 Pro, and the single Bundle ID (com.personal.linerios)[3]:

```yaml
YAML
name: LineriOS
options:
  bundleIdPrefix: com.personal
  deploymentTarget:
    iOS: "26.0"
  xcodeVersion: "26.0"
  generateEmptyDirectories: true

settings:
  base:
    SWIFT_VERSION: "6.0"
    TARGETED_DEVICE_FAMILY: "1" # iPhone only
    IPHONEOS_DEPLOYMENT_TARGET: "26.0"
    ENABLE_USER_SCRIPT_SANDBOXING: YES

targets:
  LineriOS:
    type: application
    platform: iOS
    sources:
      - path: Sources
    resources:
      - path: Resources
    info:
```

```
      path: Resources/Info.plist
      properties:
        CFBundleDisplayName: Liner
        CFBundleShortVersionString: "1.0.0"
        CFBundleVersion: "1"
        UILaunchScreen: {}
        UISupportedInterfaceOrientations:
          - UIInterfaceOrientationPortrait
        CFBundleURLTypes:
          - CFBundleURLName: com.personal.linerios.auth
            CFBundleURLSchemes:
              - linerios
        LSApplicationQueriesSchemes:
          - spotify
          - spotify-action
          - sidestore
        CADisableMinimumFrameDurationOnPhone: true # Enables 120Hz ProMotion on iPhone 14
Pro
    dependencies:
      - framework: Vendor/SpotifyiOS.xcframework
        embed: true
      - sdk: sqlite3.tbd
      - sdk: Accelerate.framework
      - sdk: AuthenticationServices.framework
      - sdk: CryptoKit.framework
    settings:
      base:
        PRODUCT_BUNDLE_IDENTIFIER: com.personal.linerios
        OTHER_LDFLAGS: "-ObjC"
        SWIFT_OBJC_BRIDGING_HEADER: "Sources/Bridge/LineriOS-Bridging-Header.h"

  LineriOSUnitTests:
    type: bundle.unit-test
    platform: iOS
    sources:
      - path: Tests
    dependencies:
      - target: LineriOS
```

The accompanying .github/workflows/build-unsigned-ipa.yml pipeline automates project synthesis, simulator unit testing, unsigned arm64 compilation, and .ipa artifact packaging on every push[3]:

YAML

```yaml
name: Build Unsigned iOS 26 IPA & Update SideStore Source

on:
  push:
    branches: [ main ]
  workflow_dispatch:

jobs:
  build-ipa:
    runs-on: macos-latest
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Select Xcode 26
        run: |
          sudo xcode-select -s /Applications/Xcode.app/Contents/Developer
          xcodebuild -version

      - name: Generate Xcode Project via XcodeGen
        run: |
          brew install xcodegen
          xcodegen generate

      - name: Run Unit Tests (Clock Interpolator, LRC/TTML Parser, SQLite Cache)
        run: |
          xcodebuild test \
            -project LineriOS.xcodeproj \
            -scheme LineriOS \
            -destination 'platform=iOS Simulator,name=iPhone 16 Pro,OS=latest' \
            CODE_SIGNING_ALLOWED=NO

      - name: Compile Unsigned Release .app for iPhone 14 Pro (arm64)
        run: |
          xcodebuild build \
            -project LineriOS.xcodeproj \
            -scheme LineriOS \
            -configuration Release \
            -sdk iphoneos \
            -derivedDataPath build \
```

```
          ARCHS=arm64 \
          ONLY_ACTIVE_ARCH=NO \
          CODE_SIGN_IDENTITY="" \
          CODE_SIGNING_REQUIRED=NO \
          CODE_SIGNING_ALLOWED=NO \
          AD_HOC_CODE_SIGNING_ALLOWED=YES

      - name: Package Unsigned .ipa Bundle
        run: |
          mkdir -p Payload
          cp -r build/Build/Products/Release-iphoneos/LineriOS.app Payload/
          zip -r9 LineriOS-unsigned.ipa Payload

      - name: Upload Unsigned .ipa Artifact
        uses: actions/upload-artifact@v4
        with:
          name: LineriOS-unsigned-ipa
          path: LineriOS-unsigned.ipa
```

Complete Repository Directory Structure

Directory / File Path	Architectural Layer & Functional Responsibility
.github/workflows/build-unsigned-ipa.yml	CI/CD Pipeline: €0 automated Xcode 26 project generator, unit test runner, and unsigned .ipa packager[3].
project.yml & sidestore-source.json	Root Config: XcodeGen declarative specification and OTA AltStore/SideStore repository manifest[3].
Vendor/SpotifyiOS.xcframework/	Vendor Binary: Official Spotify iOS App Remote SDK binary framework for local IPC[3].
Resources/Info.plist & Assets.xcassets/	App Bundle Resources: URL schemes (linerios, spotify, sidestore), 120Hz ProMotion flag, and icons[3].
Sources/App/{LinerApp,AppEnvironment}.swift	Application Root: @main SwiftUI lifecycle, URL callback router, and dependency

	injection container[3].
Sources/Bridge/LineriOS-Bridging-Header.h	Objective-C Bridge: Exposes #import <SpotifyiOS/SpotifyiOS.h> to Swift 6[3].
Sources/Domain/Models/{PlaybackSnapshot,TrackItem,LyricsPayload,ThemePreset}.swift	Domain Models: Pure Swift 6 Sendable and Codable value types[3].
Sources/Domain/Protocols/{PlaybackController,LyricsProvider,PaletteExtractor}Protocol.swift	Domain Contracts: Strict protocol interfaces decoupling UI and domain logic from external vendors[3].
Sources/Domain/Engine/{ClockInterpolator,BraccatoLyricsParser}.swift	Core Engines: 120Hz CADisplayLink monotonic drift solver and LRC/Enhanced LRC/TTML syllable parser[3].
Sources/Data/Spotify/{SpotifyPKCEAuthenticator,SpotifyAppRemoteAdapter,SpotifyWebAPIClient,HybridPlaybackCoordinator}.swift	Spotify Hybrid Data Layer: RFC 7636 PKCE auth, local IPC socket adapter, 2026-compliant REST client, and failover coordinator[3].
Sources/Data/Lyrics/{LRCLIBProvider,LocalLRCFileProvider,CompositeLyricsRepository}.swift	Lyrics Data Layer: Primary LRCLIB REST client, Files.app .lrc/.ttml importer, and 5-stage failover orchestrator[3].
Sources/Data/Visuals/VImagePaletteExtractor.swift	Visual Engine: $64 \times$ vImage downsampler + CIE LAB k-means clustering + WCAG AA contrast clamper[3].
Sources/Data/Persistence/{KeychainStore,SQLiteCacheActor,BackupMigrationService,RingBufferLogger}.swift	Persistence & Security: Hardware Keychain vault, WAL-mode sqlite3 actor, JSON backup/restore service, and in-memory diagnostic logger[3].
Sources/Presentation/Stores/{PlayerStore,LyricsStore,SearchStore,LibraryStore,ThemeManager}.swift	State Stores: @MainActor @Observable view models driving SwiftUI state updates[3].
Sources/Presentation/Components/{AdaptiveAtmosphereMeshView,LiquidGlassMiniPla	Shared UI Components: 3×3 Metal MeshGradient surface, S8 morphing

yer,DiagnosticOverlayHUD}.swift	capsule, and Windows-less floating debugger[3].
Sources/Presentation/Screens/{Welcome,AuthConfig,Home,Search,Library,NowPlaying,Lyrics,ThemeCustomizer,Settings,SystemStatusModals}View.swift	Screen Views: Complete implementations of Screens S1 through S12[3].
Tests/{ClockInterpolator,BraccatoLyricsParser,SQLiteCacheActor}Tests.swift	Automated Verification: XCTest suites executed on every GitHub Actions push[3].

Operational State Transitions, Legal SQLite Caching, and Portable Migration

Runtime Playback, Backgrounding, and Sideloading Behavior Matrix

Operational Scenario	Active Audio Owner	App & System State	Exact Technical Behavior & Recovery Flow
1. Normal Foreground Listening	Spotify iOS App (Background)[3]	LineriOS Foregrounded (120Hz)[3]	SPTAppRemote socket is open; Play/Pause/Seek execute via local IPC in <15ms; ClockInterpolator drives 120Hz lyric scrolling with zero HTTP calls[3].
2. User Locks iPhone 14 Pro Screen	Spotify iOS App (Background)[3]	LineriOS Suspended (scenePhase == .background)[3]	LineriOS immediately pauses CADisplayLink and MeshGradient shaders (0% GPU/CPU usage); Lock Screen displays Spotify's native MPNowPlayingInfo

			Center card[3].
3. User Unlocks Phone / Returns to App	Spotify iOS App (Background)[3]	LineriOS Active (scenePhase == .active)[3]	SPTAppRemote reconnects in <120ms, fetches latest playbackPosition, updates (P_anchor, T_anchor), and snaps lyrics to the exact current syllable[3].
4. Desk / MagSafe Docked Lyric Mode	Spotify iOS App (Background)[3]	LineriOS Foregrounded + isIdleTimerDisabled = true[3]	Screen stays awake indefinitely; if iPhone is unplugged and battery drops below 20%, isIdleTimerDisabled automatically resets to false to protect battery[3].
5. User Force-Kills Spotify App	None[3]	LineriOS Foregrounded[3]	SPTAppRemote fires didDisconnectWithError; LineriOS shows the "Wake Spotify" glass banner; tapping invokes SPTAppRemote.authorizeAndPlayURI(lastTrackURI), waking Spotify in background and returning focus to LineriOS[3].
6. Network Drops (Subway / Airplane	Spotify iOS App (Offline Cache)[3]	NWPathMonitor status =	Local SPTAppRemote

Mode)		.unsatisfied[3]	IPC continues working 100% offline over on-device sockets; CompositeLyricsRepository serves cached .lrc/TTML files from SQLite[3].
7. Playback Handed Off to Windows 11 PC (Spotify Connect)	Spotify Desktop App[3]	LineriOS Foregrounded (Remote Controller Mode)[3]	HybridPlaybackCoordinator detects idle local IPC while GET /v1/me/player shows active "Windows 11 Desktop"; switches to Cloud Connect Mode, polling /v1/me/player every 5.0s while interpolating 120Hz lyrics locally on the iPhone[3].
8. Incoming Call / Siri / Alarm Interruption	iOS System (Ringtone / Siri)[3]	Audio Interrupted[3]	Spotify pauses via AVAudioSession.interruptionNotification and sends isPaused = true to LineriOS; ClockInterpolator freezes at $R =$ and resumes cleanly when the interruption ends[3].
9. 7-Day Free Apple ID Certificate Expires	N/A[3]	iOS blocks launch ("LineriOS is no longer available")[3]	**Critical Rule**: Do NOT uninstall LineriOS; re-signing over the existing installation via SideStore (or

			Windows iloader) preserves the entire app sandbox (Documents/liner_cache.sqlite and Keychain tokens) intact[3].

Legal SQLite Caching and Portable JSON Backup Architecture

All structured persistence is managed by SQLiteCacheActor, a thread-safe Swift 6 actor wrapping the system C sqlite3 library at Documents/liner_cache.sqlite with PRAGMA journal_mode = WAL; enabled for non-blocking concurrent reads and writes[3].

SQLite Table / Cache Store	Exact Schema & Storage Mechanism	Legal Caching Policy	TTL & Eviction Strategy
lyrics_cache	track_uri TEXT PRIMARY KEY, provider TEXT, granularity TEXT, lines_json BLOB, plain_text TEXT, user_offset_ms INTEGER DEFAULT 0, updated_at INTEGER[3]	Permitted (LRCLIB open database & user-imported .lrc/.ttml files)[3].	Permanent (Never auto-evicted; ~4KB/song = 10,000 songs occupy <40MB); user_offset_ms is preserved across re-fetches[3].
palette_cache	artwork_url TEXT PRIMARY KEY, bg_hex_0 TEXT, bg_hex_1 TEXT, bg_hex_2 TEXT, bg_hex_3 TEXT, accent_hex TEXT, extracted_at INTEGER[3]	Permitted (Computed hex color vectors derived locally on-device)[3].	Permanent (~180 bytes per album; eliminates redundant image decoding)[3].
metadata_cache	spotify_uri TEXT PRIMARY KEY,	Permitted Ephemeral Cache	7-Day Rolling TTL (Refreshed

	entity_type TEXT, payload_json BLOB, cached_at INTEGER[3]	(Offline display of playlists and recent tracks)[3].	automatically when online; purged upon account logout)[3].
artwork_disk_cache	Dedicated URLCache(memory Capacity: 30_000_000, diskCapacity: 150_000_000)[3]	Permitted Temporary Display Cache[3].	Automatic LRU eviction at 150 MB or one-tap purge in S11 Settings[3].
Raw Spotify Audio Streams	**NONE**[3]	**Strictly Prohibited by Spotify Developer Terms**[3].	Never intercepted, buffered, or written to disk under any circumstance[3].

To protect user customizations across device resets or Apple ID changes, BackupMigrationService.swift exports and restores a deterministic portable JSON archive (LinerBackupArchive v1) via UIDocumentPickerViewController[3]. For security, OAuth access_token and refresh_token credentials are excluded from the JSON export[3]. Restoring an archive executes inside a single atomic SQLite transaction (BEGIN IMMEDIATE TRANSACTION ... ON CONFLICT(track_uri) DO UPDATE SET ... COMMIT), merging cached lyrics and custom per-track millisecond offsets without duplicating records[3]:

```
JSON
{
  "schemaVersion": 1,
  "exportedAt": "2026-10-24T14:30:00Z",
  "deviceModel": "iPhone15,2",
  "preferences": {
    "selectedTheme": "midnight",
    "customAccentHex": "#38BDF8",
    "adaptiveAtmosphereEnabled": true,
    "meshFrameRate": 60,
    "keepAwakeInLyricsMode": true
  },
  "customTrackOffsets": [
    { "trackURI": "spotify:track:4cOdK2wGLETKBW3PvgPWqT", "offsetMs": -200 }
```

```
"cachedLyrics"
    "trackURI" "spotify:track:4cOdK2wGLETKBW3PvgPWqT"
    "granularity" "syllableSynced"
    "lines"
```


Hardware Guardrails, Zero-Secret Security, Verification, and Implementation Roadmap

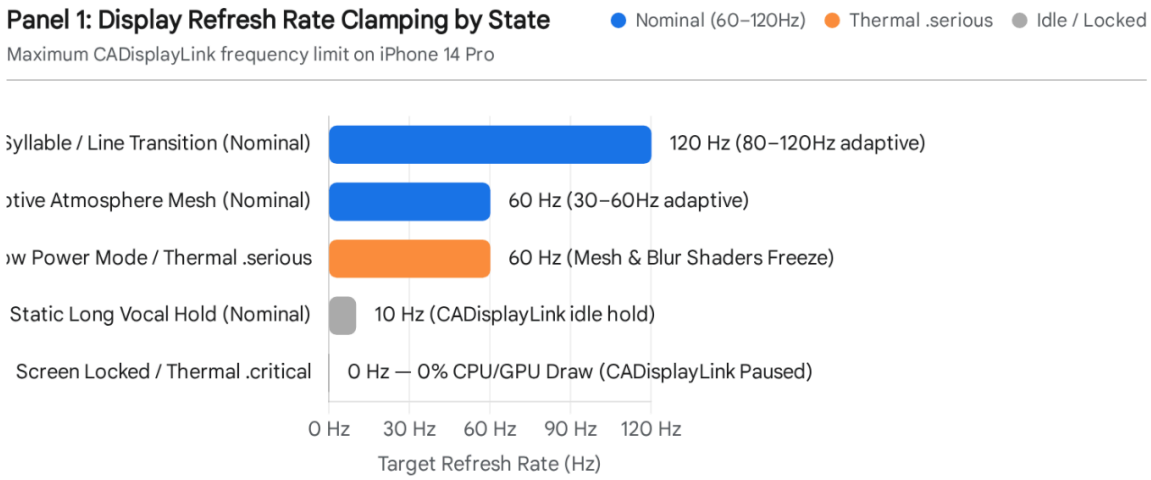
Performance, 120Hz ProMotion Pacing, and Thermal Guardrails

The iPhone 14 Pro's A16 Bionic and Super Retina XDR ProMotion display scale dynamically between 10Hz and 120Hz[3]. Without explicit CAFrameRateRange hints and ProcessInfo.processInfo.thermalState observers, continuous shader evaluation and text blurring can induce thermal throttling[3].

ProMotion Frame-Rate Clamping and Sub-15ms Local Execution Protect Battery and Thermal Headroom

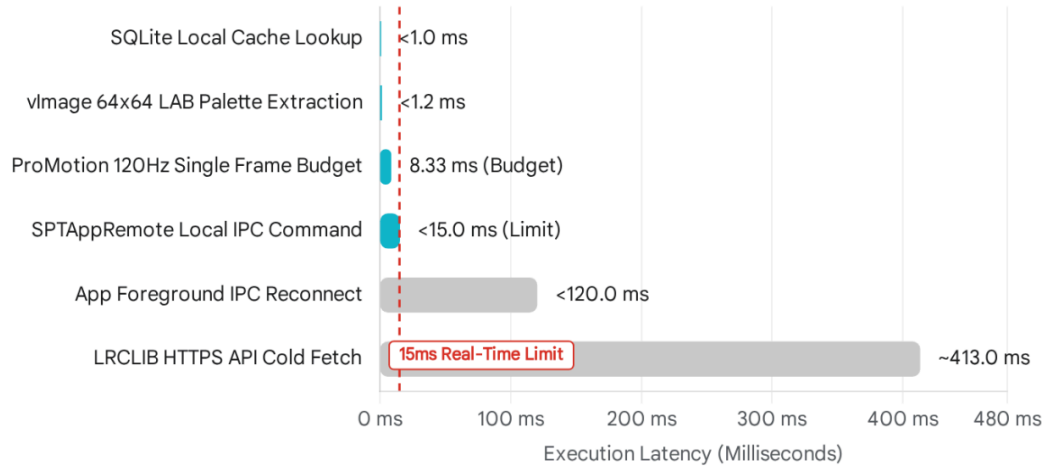
Panel 1: Display Refresh Rate Clamping by State

Maximum CADisplayLink frequency limit on iPhone 14 Pro



Panel 2: Subsystem Single-Operation Execution Latency

Runtime benchmarks vs. 15ms real-time rendering deadline



Dynamic refresh-rate throttling across iPhone 14 Pro operational states (top) and single-operation execution latencies across core subsystems (bottom). By stepping down CADisplayLink from 120Hz during active syllable transitions to 10Hz during vocal holds and 0Hz when backgrounded, the engine maintains flagship fluidity with minimal thermal draw.

Data sources: Liner Desktop Architecture, Braccato Core Performance Benchmarks

Subsystem	Normal Operation (Battery > 20%, Thermal .nominal/.fair)	Eco / Low Power Mode (isLowPowerMode Enabled or Thermal .serious)	Critical Thermal State (.critical) or App Backgrounded
Lyric Scroll & Syllable Fill (CADisplayLink)	CAFrameRateRange(minimum: 80, maximum: 120, preferred: 120) during active syllable/line transitions; drops to 10Hz during static vocal holds[3].	Clamped to CAFrameRateRange(minimum: 30, maximum: 60, preferred: 60); distance-based Gaussian line blur disabled (blur(radius: 0))[3].	CADisplayLink.isPaused = true (0 Hz / 0% CPU draw)[3].
Adaptive Atmosphere (MeshGradient)	3×3 Metal MeshGradient control points oscillate smoothly at 30–60Hz[3].	Frozen into a static 3×3 MeshGradient (animates for 0.6s only on track change, then holds static at 0% GPU)[3].	Replaced by solid theme base color (#000000 in Obsidian)[3].
Artwork Palette Extraction (vImage)	Runs once per unique artworkURL on a background .utility task at $64 \times$ resolution ($<$)[3].	Identical ($<$), served permanently from SQLite palette_cache thereafter[3].	Deferred until application returns to foreground[3].
Spotify State Synchronization	Push-driven via SPTAppRemotePlayerStateDelegate (0 HTTP polls); polls GET /v1/me/player every 5.0s only in Cloud Connect mode[3].	Cloud Connect poll interval extended to 10.0s[3].	All timers and network pollers suspended[3].

Privacy, RFC 7636 PKCE, and Zero-Secret Repository Security

Because the GitHub repository is public to unlock unlimited free macos-latest Xcode 26 build minutes, the security architecture enforces a strict zero-secret rule: no client_secret, no hardcoded client_id, no .p12 signing certificates, and no .mobileprovision files ever exist in the git tree[1]. Code signing happens 100% locally on the iPhone 14 Pro inside SideStore after downloading the unsigned .ipa[3].

Authentication implements **RFC 7636 OAuth 2.0 PKCE** (SpotifyPKCEAuthenticator.swift), generating a 64-byte cryptographically random code_verifier via SecRandomCopyBytes and computing

$$\text{code_challenge} = \text{base64URLEncode}(\text{SHA256}(\text{ASCII}(\text{code_verifier})))$$

via Apple's CryptoKit[3]. The handshake executes inside ASWebAuthenticationSession with callback scheme linerios://auth-callback and strict CSRF state validation[3]. Resulting access_token, refresh_token, token_expiry, and client_id secrets are stored in KeychainStore.swift using kSecClassGenericPassword with protection class kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly—ensuring hardware encryption inside the A16 Secure Enclave and preventing token extraction via unencrypted backups[3]. Outbound network traffic is restricted strictly to *.spotify.com (accounts.spotify.com, api.spotify.com, i.scdn.co) and lrclib.net with zero third-party telemetry[3].

Automated CI Verification and Physical iPhone 14 Pro Acceptance Criteria

Verification Layer	Suite / Milestone Check	Target Component	Exact Assertions & Acceptance Criteria Verified
CI Unit Test Suite 1	ClockInterpolatorTests	ClockInterpolator.swift	(1) At $R =$, advancing monotonic clock by 250.0ms advances t_{now} by exactly 250ms; (2) at $R =$, t_{now} freezes; (3) userOffsetMs = -300 shifts t_{now} by -300ms; (4) drift $<$ is ignored while drift

			> slews smoothly over 10 frames[3].
CI Unit Test Suite 2	BraccatoLyricsParserTests	BraccatoLyricsParser.swift	(1) Parses standard .lrc ([01:12.45] Line) into LyricLine arrays; (2) parses Enhanced LRC (<00:15.20>Word1 <00:15.65>Word2) and TTML into LyricWordToken spans with exact syllable progress math; (3) handles malformed/duplicate timestamps and [au: Instrumental] tags without crashing[3].
CI Unit Test Suite 3	SQLiteCacheActorTests	SQLiteCacheActor.swift	(1) Inserts and retrieves LyricsPayload and PaletteHex concurrently across 20 tasks without SQLITE_BUSY locks; (2) user_offset_ms updates persist across database reopen; (3) JSON backup export and UPSERT restore round-trip 100% identically[3].
CI Unit Test Suite 4	Spotify2026SchemaTests	SpotifyWebAPIClient.swift	(1) Decodes Feb/Mar 2026 Dev Mode /v1/playlists/{id}/ite

			ms JSON where tracks is renamed to items and track is renamed to item; (2) gracefully decodes tracks omitting popularity and external_ids[3].
Physical Device Check 1	Sideload & Slot Audit	SideStore + LocalDevVPN	Install LineriOS-unsigned.ipa on iPhone 14 Pro; confirm active Bundle IDs = 2 (SideStore + com.personal.linerios), leaving 1 free slot spare[3].
Physical Device Check 2	PKCE & Local IPC Latency	S2 Auth & S7 Transport	Paste client_id in S2, complete PKCE, force-close and relaunch directly into S3 (<15ms); verify Play/Pause/Skip respond in <25ms via SPTAppRemote with 0 HTTP calls[3].
Physical Device Check 3	120Hz ProMotion & Fallback	S9 Lyrics & S11 HUD	Enable DiagnosticOverlayHUD, verify 118–120 FPS during lyric transitions; verify automatic LRCLIB sync on a mainstream Romanian track, .lrc file import on an underground track, and full offline

			survival in Airplane Mode[3].

Long-Term Maintenance Runbook and Risk Mitigation Matrix

To eliminate manual weekly maintenance, a native iOS **Personal Automation** is configured in the built-in Apple Shortcuts app on the iPhone 14 Pro triggered daily at 03:00 AM (or when connected to Wi-Fi and charger): the automation connects LocalDevVPN, waits 3 seconds, and executes the SideStore *"Refresh All Apps"* Shortcut action[3]. This resets the 7-day Personal Team certificate back to 7 days remaining overnight without touching the Windows 11 PC[3]. Before installing any iOS point upgrade (e.g., iOS 26.5 → iOS 26.6 / iOS 27), the operator verifies LocalDevVPN and Anisette compatibility on docs.sidestore.io and exports a 3-second JSON backup from S11 Settings to iCloud Drive[3].

Risk ID	Concrete Operational Risk	Likelihood & Impact	Engineered Mitigation Built Into Blueprint
R1	SideStore Error 1009 / 3013: Exceeding 10 App ID registrations within a rolling 7-day period[3].	Medium / High[3]	project.yml compiles only 1 target (com.personal.linerios) with zero App Extensions by default; refreshing an existing app consumes 0 new App IDs[3].
R2	7-Day certificate expires while traveling without the Windows 11 laptop[3].	Low–Med / Medium[3]	Never delete the expired app; re-signing over the existing installation via SideStore (or Windows iloader upon return) restores launch access with 100% of SQLite and Keychain data intact[3].

R3	SideStore Error 3021 (Invalid Anisette Data) during on-device certificate refresh[3].	Low / Low[3]	Switch the active Anisette server URL inside *SideStore → Settings → Anisette Server* to an alternate listed public server and retry[3].
R4	SPTAppRemote drops local IPC socket after iOS suspends background audio processes[3].	High / Low[3]	HybridPlaybackCoordinator automatically routes commands via Spotify Web API while surfacing a one-tap "Wake Local Spotify" pill that pings spotify: and restores IPC in <0.8s[3].
R5	Spotify Web API HTTP 429 Too Many Requests during rapid playlist browsing[3].	Low / Medium[3]	SpotifyWebAPIClient enforces a strict token-bucket limiter (max 2 req/sec), caches playlist items in SQLite for 7 days, and honors the Retry-After header via S12 countdown pills[3].
R6	Track metadata suffixes (- Remastered, (feat. X)) cause LRCLIB 404 on exact match[3].	High / Low[3]	CompositeLyricsRepository runs regex title normalization stripping feature/remaster tags and falls back to GET /api/search filtered by ± 2.5s duration tolerance[3].

R7	Spotify Client ID owner account downgrades from Premium to Free (403 Forbidden)[3].	User-Dependent / High[3]	S2 and S12 detect the 403 Development Mode policy response, explain the March 2026 Premium owner requirement, and maintain full access to offline cached lyrics[3].

Milestone-by-Milestone Implementation Roadmap

Executing the build follows a strict 14-day, five-milestone sequence designed to validate the Windows-to-iPhone hardware pipeline on Day 1 before layering domain complexity[3]:

Milestone Phase	Target Timeline	Core Engineering Deliverables	Verification Gate (Minimum Viable Exit Criteria)
Milestone 0: Pipeline Proof ("Hello Glass")	Day 1 (Mandatory First Step)[3]	Initialize public GitHub repository with project.yml, .github/workflows/build-unsigned-ipa.yml, and a 50-line SwiftUI view rendering a single iOS 26 .glassEffect() card on an Obsidian #000000 background[3].	GitHub Actions compiles LineriOS-unsigned.ipa in ~3 minutes; SideStore + LocalDevVPN installs it on the physical iPhone 14 Pro; touch input responds at 120Hz[3].
Milestone 1: Zero-Secret PKCE & Hybrid Engine	Days 2–4[3]	Implement KeychainStore, SpotifyPKCEAuthenticator (ASWebAuthenticationSession), SpotifyWebAPIClien	On-device PKCE handshake succeeds; DiagnosticOverlayHUD inspects live IPC and HTTP traffic directly on the

		t (2026 /items & /me/library endpoints), SpotifyAppRemoteAdapter, and Screens S1, S2, and S11 DiagnosticOverlayHUD[3].	iPhone screen without a Mac; local IPC transport responds in <15ms[3].
Milestone 2: 120Hz Clock, LRCLIB & SQLite	Days 5–7[3]	Implement SQLiteCacheActor (WAL mode), ClockInterpolator (CADisplayLink), BraccatoLyricsParser (LRC + Enhanced LRC/TTML), LRCLIBProvider, .lrc file import, and S9 LyricsView[3].	120Hz syllable fill, active-line glow, distance Gaussian blur, and $\pm 100\text{ms}$ per-track offset stepper operate at 118–120 FPS with permanent SQLite offline caching[3].
Milestone 3: Liquid Glass UI & Atmosphere	Days 8–10[3]	Implement VImagePaletteExtractor ($64 \times$ LAB k-means + WCAG AA clamper), AdaptiveAtmosphereMeshView (3×3 MeshGradient), and Screens S3 (HomeView), S4/S5 (SearchView), S6 (LibraryView), S7 (NowPlayingView), S8 (LiquidGlassMiniPlayer), and S10 (ThemeCustomizerView)[3].	S8 Mini-Player smoothly morphs into S7 via .glassEffectID; 4 theme presets and real-time artwork mesh adaptation pass WCAG AA contrast checks ($> 5.52:$)[3].
Milestone 4:	Days 11–14[3]	Implement	All 4 CI unit test

Backup, Guardrails & Acceptance		BackupMigrationService (.json export/import), thermal/low-power CAFrameRateRange throttling, S12 error/offline banners, and the nightly iOS Shortcuts SideStore refresh automation[3].	suites and all physical iPhone 14 Pro acceptance checks pass; nightly automated SideStore refresh resets the 7-day certificate hands-free[3].

Lucrări citate

1. phase 1A.txt
2. prompt.md
3. phase 1B.txt
4. tryliner/desktop: Liner is a desktop player for Windows and ... - GitHub, https://github.com/tryliner/desktop
5. Download the Liner Music App, https://liner01.online/download
6. Liner - GitHub, https://github.com/tryliner
7. GitHub - better-lyrics/braccato, https://github.com/better-lyrics/braccato
8. GitHub - better-lyrics/kawarp: Fluid animated backgrounds powered, https://github.com/better-lyrics/kawarp
9. Kawarp, https://kawarp.vercel.app/
10. @kawarp/core CDN by jsDelivr - A CDN for npm and GitHub, https://www.jsdelivr.com/package/npm/@kawarp/core
11. com.eza.spicyex/NOTICE at main · Xposed-Modules-Repo ... - GitHub, https://github.com/Xposed-Modules-Repo/com.eza.spicyex/blob/main/NOTICE